

Muhammad Farooqi — Interview Prep Cheat Sheet

Lincoln Electric Automation — Robotics Software Engineer Intern

Q1: What is tack welding and what did the robot do before you built this?

Interviewer question:

Walk me through what tack welding actually is, what the robot was doing before you built this, and what 'end-to-end' means specifically in this context.

Model Answer:

Tack welding is stationary spot welding — the robot holds position and fires a short weld to temporarily hold two pieces of metal together before the actual full welding pass takes place. Full welding is continuous — the robot moves along a seam and lays down a full weld bead. Tack welding secures the pieces first so they don't shift during full welding.

Before I built this, the workflow required a separate manual welder — operators would make the stationary tack welds manually, then switch over to the cobot arm for the full weld. That switch between two separate pieces of equipment cost 5-7 seconds per tack and 2-3 minutes per part across the two production cells.

End-to-end means I owned the full flow: from the teach pendant UI where operators define their wire preferences, tack size, voltage, and duration — all the way through to the arm executing the weld safely. Everything centralized into one interface, one device, one workflow.

Q2: KAREL vs RAPID — what's the difference and how did you handle both?

Interviewer question:

You implemented the same tack welding logic on both Fanuc (KAREL) and ABB (RAPID). Walk me through what each language is and what the biggest technical difference was between the two implementations.

Model Answer:

KAREL is Fanuc's proprietary Pascal-based language. You write .kl files with a header declaring variables, then a BEGIN/END block with your logic. Motion instructions reference position registers (PR[1], PR[2]) and welding parameters — voltage, wire feed speed, tack duration — get stored in numerical registers (R[1], R[2] etc) that the program reads at runtime. The operator sets those register values on the teach pendant before running, which makes the program flexible without touching the code.

RAPID is ABB's language — more modern and closer to a high-level language. It's organized into modules (like files) containing routines (like functions). A main routine is the entry point. RAPID was already integrated in our codebase which made it more straightforward. Instead of numbered registers, you work with named variables and structured routines that interface with the controller API.

The core welding logic — tack size, duration, voltage — was identical on both arms. The difference was how you communicate that to each controller. On Fanuc I was writing directly to numbered registers in

KAREL. On ABB I was calling RAPID routines through the controller API. Same logic, completely different plumbing.

■ *If asked which was harder: KAREL was harder — the register-based interface is lower level and less intuitive. RAPID felt more like writing normal code.*

Q3: What is a teach pendant and how did operators use it?

Interviewer question:

What is a teach pendant and how did operators actually use it to trigger your tack welding program on both arms?

Model Answer:

A teach pendant is the handheld controller tethered to the robot arm — like a tablet physically connected to the robot. It has two modes: teach mode where you can jog the arm to positions safely, and run mode where you execute programs.

On the Fanuc side, the operator would use the pendant to set register values for tack size, voltage, and wire feed speed, then press a dedicated button to run the KAREL program. On the ABB side they'd select the RAPID routine and hit run. The key thing I built was making this process clean — instead of operators having to navigate deep pendant menus to set individual register values, our interface exposed only the parameters they needed in a structured way.

Q4: What parameters define a tack weld?

Interviewer question:

What are the actual parameters that define a tack weld and how did your program expose them to the operator?

Model Answer:

Three parameters define a tack: duration (how long the arc fires — typically fractions of a second), voltage (arc intensity — controls heat and penetration), and wire feed speed (how fast filler wire is fed into the weld pool — controls bead size). Together these three define the size and strength of the tack.

We supported 3 tack sizes — small, medium, large — each with a predefined set of these parameters. The operator selected the tack size on the pendant and the program loaded the corresponding register values automatically. This is what removed the manual welder setup step — all parameter configuration happened in one place.

Q5: Motion control and TypeScript — what did that actually look like?

Interviewer question:

Your resume mentions motion control and weld parameter interfaces in TypeScript using functional programming. What does TypeScript have to do with robot arm control?

Model Answer:

The motion control logic and weld parameter interfaces weren't written directly in KAREL or RAPID — they were defined in TypeScript as part of our broader software stack, then communicated to the robot controllers via Controller APIs. TypeScript sat in the middle: it defined the tack configurations, validated parameters, and sent the right values to the correct controller depending on whether we were targeting the Fanuc or ABB arm.

Functional programming in this context meant we built pure functions for each tack configuration — no side effects, predictable outputs — which made testing straightforward. We validated through 100+ virtual welds in RobotStudio (ABB's simulator) and RoboGuide (Fanuc's simulator) before touching the physical arms.

■ *If asked why TypeScript for robotics: it was already the language of the broader codebase. The Controllers had APIs we could call — TypeScript was the orchestration layer on top.*

Q6: Why 100+ virtual welds — what were you actually testing?

Interviewer question:

You validated through 100+ virtual welds in RobotStudio and RoboGuide. Why 100+? What were you actually running each time, and how did you know when you had enough confidence to deploy on the physical production cells?

Model Answer:

RobotStudio is ABB's simulation environment — it lets you simulate the full robot cell including the arm, workpiece, and controller virtually. RoboGuide is Fanuc's equivalent. You can run your RAPID or KAREL programs and see exactly how the arm moves before touching the physical robot.

The two main things that could break were motion path collisions and weld parameter mismatches. For collisions — when you add a tack weld, the arm holds position, fires the weld, then retracts before continuing the motion plan. If the retract path wasn't clear, the arm could collide with the workpiece or fixture. RoboGuide and RobotStudio let you visualize exactly where the TCP (tool center point) is at every moment, so we could catch any collision before it happened on the real cell.

For weld parameters — if voltage or wire feed speed was set incorrectly in the registers, you'd either get a tack that was too weak and didn't hold, or you'd burn through the material. We ran 100+ welds across all 3 tack sizes on both arms to verify each size produced consistent expected results.

We did catch something — early on there was a retract path issue on one of the Fanuc configurations where the arm would clip the fixture on the way out. We caught it in simulation, adjusted the retract waypoint, and re-validated before touching the physical cell. That's exactly why you do virtual validation first.

■ *Always say you caught something — 'everything passed cleanly' sounds like you didn't test hard enough. The retract path collision is your concrete example. This maps directly to Multiply Labs' simulation and robot validation requirement.*

Background Knowledge — Study These

KAREL (Fanuc)

Pascal-based proprietary language for Fanuc robots. Programs are .kl files with a header (variable declarations) and BEGIN/END block (logic). Motion uses position registers PR[1], PR[2] etc. Weld parameters stored in numerical registers R[1], R[2] etc — operator sets values on teach pendant, program reads them at runtime. I/O signals control weld on/off. Lower level and less intuitive than RAPID.

RAPID (ABB)

ABB's robot programming language — more modern, closer to high-level languages. Organized into modules (files) containing routines (functions). Main routine is entry point. Named variables instead of numbered registers. Already existed in the Lincoln codebase. Felt more natural to work with than KAREL.

Teach Pendant

Handheld controller tethered to the robot arm. Two modes: teach mode (jog arm to positions safely, define waypoints) and run mode (execute programs). Operators use it to set parameters and trigger program execution. On Fanuc: set register values then run KAREL program. On ABB: select RAPID routine and hit run.

Tack Welding Parameters

Duration (how long arc fires — fractions of a second), Voltage (arc intensity — controls heat and penetration), Wire feed speed (how fast filler wire feeds into weld pool — controls bead size). These three together define tack size and strength. We supported 3 tack sizes (small, medium, large) each with predefined parameter sets.

RobotStudio (ABB) and RoboGuide (Fanuc)

Simulation environments for each robot platform. Full virtual robot cell — arm, workpiece, controller. Run programs virtually before touching physical hardware. Used to validate motion paths, weld parameters, edge case handling, and collision avoidance. Equivalent to Gazebo/Isaac Sim in ROS2 world — same concept, proprietary toolchain.